Randomized Numerical Linear Algebra for the Sciences

Tyler Chen (陈泰乐)

March 10, 2026

https://research.chen.pw/talks/NYUSH26.pdf

Overview

History:

- PhD in Applied Math at University of Washington
- Assistant Professor / Courant Instructor at Tandon CS and Courant Math
- Applied Research Lead at Global Technology Applied Research, JPMorganChase

Research: I design and analyze fast and theoretically justified (randomized) algorithms for core linear algebra tasks.

- My work blends key areas of computer/data science such as numerical analysis and theoretical computer science.

Goal: My broad goal is to develop tools to support the advancement of knowledge in current scientific applications.

Misc: I really like working with students / teaching.

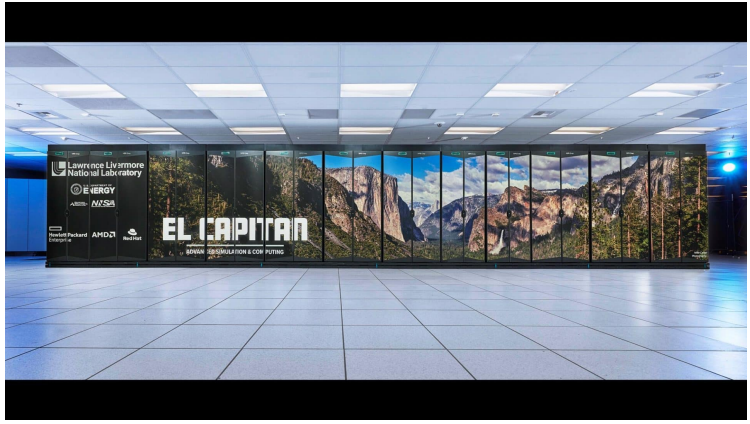
Numerical Linear Algebra

NLA has been a core part of computer science since the outset (4 Turing awards!!)

NLA is about "doing linear algebra on computers". Centered on designing/understanding algorithms to efficiently and accurately do core linear-algebra tasks:

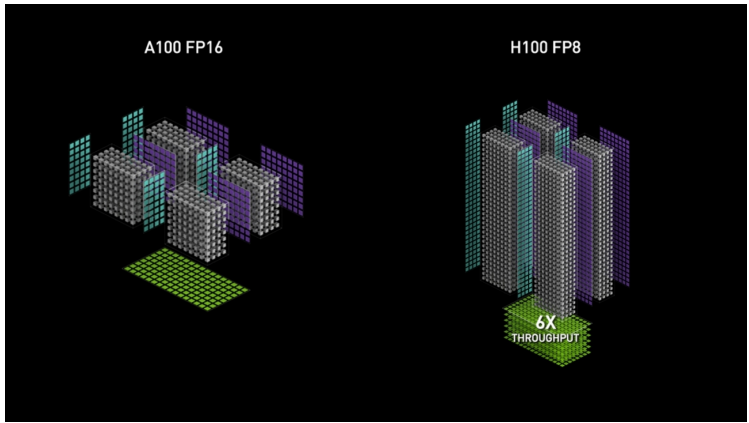
- compute $\mathbf{AB} + \mathbf{C}$
- factor $\mathbf{A} = \mathbf{LU}$ or $\mathbf{A} = \mathbf{QR}$
- solve $\mathbf{Ax} = \mathbf{b}$ or $\min_{\mathbf{x}} \|\mathbf{b} - \mathbf{Ax}\|$
- compute eigenvalues/SVD
- compute $f(\mathbf{A})$ or $f(\mathbf{A})\mathbf{b}$
- compute $\log(\det(\mathbf{A}))$
- etc.

Linear-algebra drives modern computing



43k CPUs + 43k GPUs + fast interconnects → simulate physics (using NLA)

Linear-algebra drives modern computing



NVidia makes devices to do matrix-operations fast

Numerical Linear Algebra across computing

Machine Learning

- Training/optimization
- Inference
- Model compression

Scientific Computing

- PDE solvers
- Quantum simulations
- Climate modeling
- Molecular dynamics

Data Science

- PCA/clustering/regression
- Compression
- Netflix problem
- Page rank

Finance

- Portfolio optimization
- Risk assessment
- Option pricing
- Covariance estimation

Randomized Numerical Linear Algebra

Recently, there has been a widespread paradigm shift: randomization is now a critical tool in algorithm design (e.g. Stochastic Gradient Descent enabled NN/LLMs)

RandNLA aims to develop randomized algorithms for linear algebra tasks.

- randomized algorithms have provided orders-of-magnitude speedups in computation, and the best theoretical running times
- the field's success is due to a joint effort of theoretical computer science and numerical analysis communities (regular joint workshops / events)

Representative work

Machine learning/data science

- Theoretical guarantees for algorithms widely used in ML
  ICML 21 long presentation, IMA J. Num. Analysis 24, NeurIPS 25, …
- Krylov subspace methods for low-rank approx/matrix functions
  SIMAX 23, NeurIPS 24 spotlight, SISC 25, SODA 26, …

Quantum Physics/Chemistry

- Algorithms for quantum theromodynamics
  J. Comp. Phys. 22/24, SISC 24, …

Operator Learning

- What can we learn about (structured) matrices from matrix-vector products?
  SIMAX 25/26, SODA 25, COLT 26 submission, …

Classical NLA

- Algorithms for HPC/GPU/etc.
  SISC 21/22/24, PDSEC 26, …
- Krylov subspace methods for linear systems
  SIMAX, SISC, ETNA, Num. Alg., …

Representative work

Machine learning/data science

- Theoretical guarantees for algorithms widely used in ML
  ICML 21 long presentation, IMA J. Num. Analysis 24, NeurIPS 25, …
- Krylov subspace methods for low-rank approx/matrix functions
  SIMAX 23, NeurIPS 24 spotlight, SISC 25, SODA 26, …

Quantum Physics/Chemistry

- Algorithms for quantum theromodynamics
  J. Comp. Phys. 22/24, SISC 24, …

Operator Learning

- What can we learn about (structured) matrices from matrix-vector products?
  SIMAX 25/26, SODA 25, COLT 26 submission, …

Classical NLA

- Algorithms for HPC/GPU/etc.
  SISC 21/22/24, PDSEC 26, …
- Krylov subspace methods for linear systems
  SIMAX, SISC, ETNA, Num. Alg., …

Today's talk: operator learning

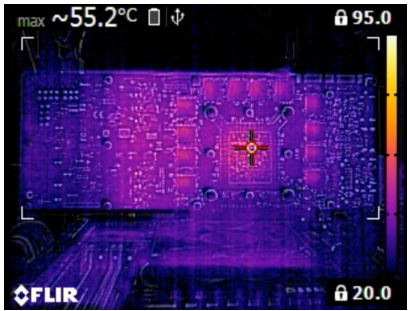
Today, I'll talk about an line of work on operator learning, which uses RandNLA to solve open problems in the emerging field of Scientific Machine Learning.

> **Operator learning** aims to discover or approximate an unknown operator (map between functions), which is often the solution operator for a partial differential equation (PDE).

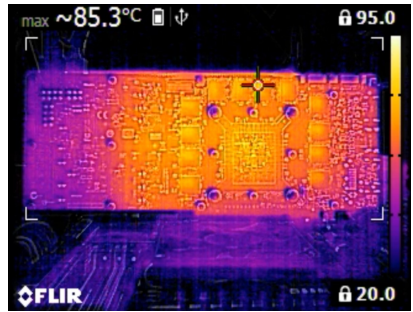
Note: I'll mostly be speaking about the theoretical side of things. There are also a very concrete applications of this line of work to speeding up real-world computation via preconditioning (ask me at the end!)

Operators describe physics

Physical processes often map a function f to a function u.



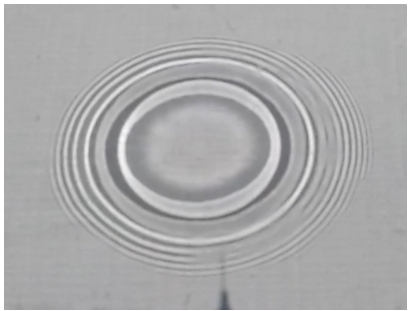
f



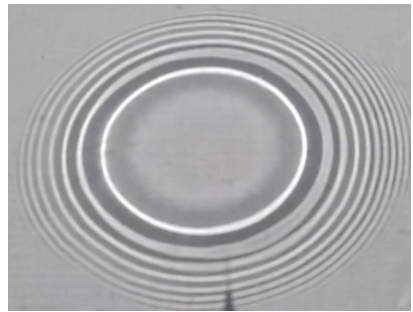
u

Operators describe physics

Physical processes often map a function f to a function u.



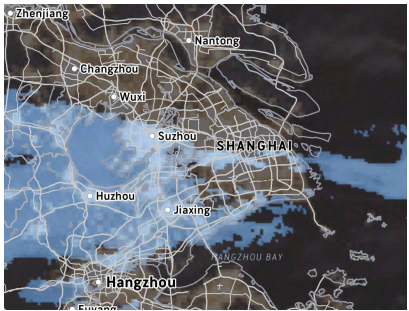
f



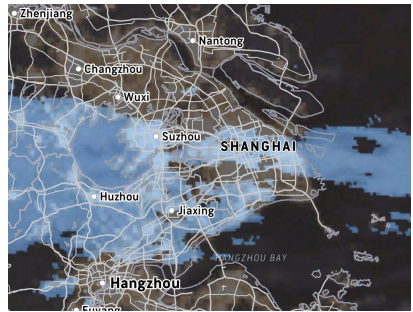
u

Operators describe physics

Physical processes often map a function f to a function u.



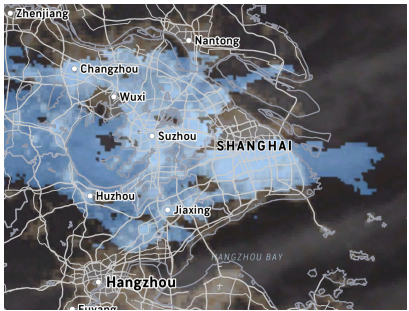
f



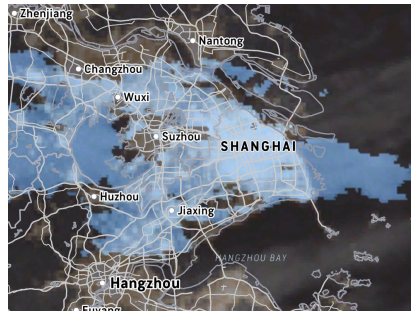
u

Operators describe physics

Physical processes often map a function f to a function u.



f



u

Operator Learning

Setting: Define underlying operator taking functions f to functions u.

Goal: Learn about operator from observed input-output pairs: $(f_1, u_1), \dots, (f_m, u_m)$.

Things to think about:

- What types of operators can we learn?
- Do we need to be able to choose f_i? What if it's randomly generated?
- How much data do we need?
- Are there limits on what can be learned?

Mapping operator learning to NLA

Q: How can we use our expertise in RandNLA to advance operator learning/SciML?

A: Distill to simple case: linear operator acting on finite dimensional spaces.

- Continuous functions become vectors and the operator becomes a matrix!
- Even this simple case is rich enough to capture a lot of relevant scientific problems and has proved hard to make progress in.

New goal: What can we learn about a matrix from a few matrix-vector products (matvecs) with $\mathbf{A}$?

- a few matvecs: $(\mathbf{x}_1, \mathbf{A}\mathbf{x}_1), \ldots, (\mathbf{x}_m, \mathbf{A}\mathbf{x}_m)$
- $\mathbf{x}_i$ may or may not be chosen adaptively based on previous outputs

Matrix Approximation

Structured Matrix Approximation.

Fix some family S of matrices. How many matrix-vector products to (unknown, fixed, arbitrary) $\mathbf{A} \in \mathbb{R}^{n \times n}$ are required to learn (a parameterization of) $\widetilde{\mathbf{A}} \in S$ such that

$$\|\mathbf{A} - \widetilde{\mathbf{A}}\|_F \leq \Gamma \cdot \min_{\mathbf{X} \in S} \|\mathbf{A} - \mathbf{X}\|_F.$$

Interesting for $\Gamma = (1+\varepsilon)$, $\Gamma = 2$, $\Gamma = \log(n)$, $\Gamma = n^{0.01}$, etc.

Past work: A lot of work on low-rank approximation.

My work: sparse matrices (SIMAX 2025), hierarchical matrices (SODA 2025, SIMAX 2026), general families (COLT 2026 submission), butterfly matrices (upcoming), …

Warm up: Low-rank matrices

Let $S = \{\text{rank-}k \text{ matrices}\}$. Lots of work on this problem![1]

The Randomized SVD (RSVD) is a well-known algorithm for obtaining an approximation to $\mathbf{A}$ from S:

1. Sample Gaussian matrix $\mathbf{\Omega}$ with $m/2$ columns
2. Form $\mathbf{Q} = \text{orth}(\mathbf{A\Omega})$
3. Compute $\mathbf{X} = \mathbf{Q}^\mathsf{T}\mathbf{A}$ (minimize: $\|\mathbf{A} - \mathbf{QX}\|_\mathsf{F}$)
4. Output $\widetilde{\mathbf{A}} = \mathbf{Q}[\![\mathbf{X}]\!]_k$

Theorem. When $m = O(k/\varepsilon)$,

$$\|\mathbf{A} - \widetilde{\mathbf{A}}\|_\mathsf{F} \le (1+\varepsilon) \cdot \min_{\mathbf{X}\text{rank-}k} \|\mathbf{A} - \mathbf{X}\|_\mathsf{F}.$$

[1] Sarlos 2006; Clarkson and Woodruff 2009; Halko, Martinsson, and Tropp 2011; Tropp and Webber 2023, etc.

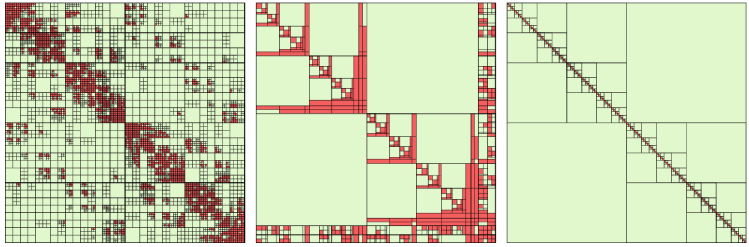
Low-rank is understood, but not rich enough

Decades of work mean we know a LOT about low-rank approximation, and have nearly-optimal algorithms.

However, matrices arising from scientific applications aren't always low-rank!!

Hierarchical matrices

Today, S will be some family of hierarchical matrices.



example classes: hierarchical off-diagonal low-rank (HODLR), hierarchical semi-seperable (HSS), $\mathcal{H}^1$, $\mathcal{H}^2$, hierarchical off-diagonal butterfly, etc.

Argonne NATIONAL LABORATORY

BERKELEY LAB
Lawrence Berkeley National Laboratory

Lawrence Livermore National Laboratory

ButterflyPACK
Fast PDE solvers and transforms

Motivation: better PDE learning

2024 SIAM Linear Algebra Best Paper Prize winner asks:[2]

Q: Can we efficiently obtain a near-optimal HODLR approximation from matrix-vector products?

Resolving this immediately gives better algorithms for learning hyperbolic PDEs.

> **Theorem** (Chen, et. al., SODA 2025). Yes!!!

[2] Boullé and Townsend 2022.

Motivation: better PDE learning

2024 SIAM Linear Algebra Best Paper Prize winner asks:[2]

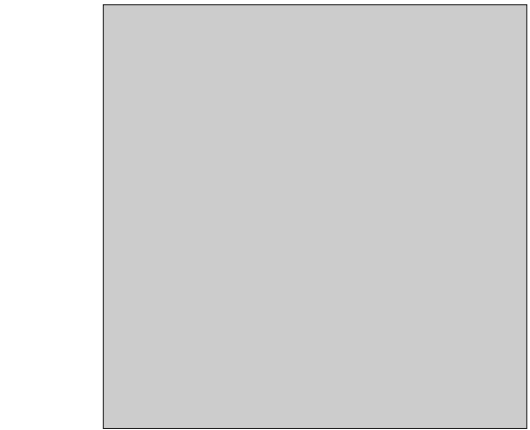
Q: Can we efficiently obtain a near-optimal HODLR approximation from matrix-vector products?

Resolving this immediately gives better algorithms for learning hyperbolic PDEs.

> **Theorem** (Chen, et. al., SODA 2025)**.** Yes!!!

[2]Boullé and Townsend 2022.

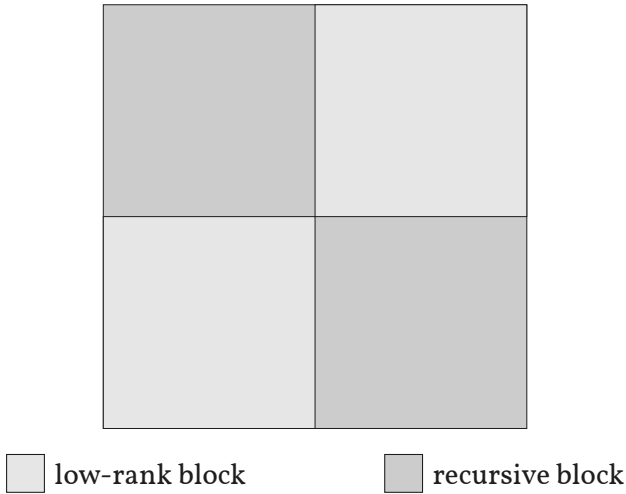
HODLR matrices



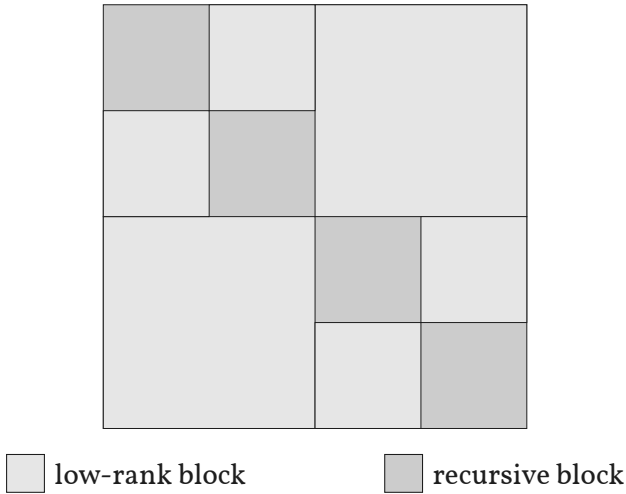
low-rank block

recursive block

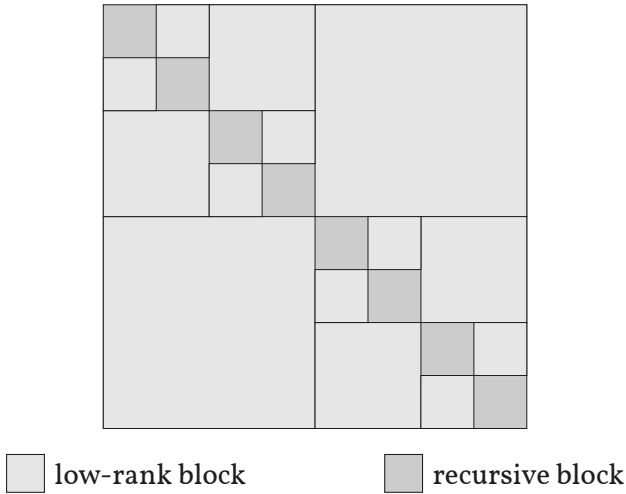
HODLR matrices



HODLR matrices



HODLR matrices



HODLR Matrices

Definition. Fix a rank parameter k. We say a $n \times n$ matrix $\mathbf{A}$ is HODLR(k) if $n \leq k$ or $\mathbf{A}$ can be partitioned into $(n/2) \times (n/2)$ blocks

$$\mathbf{A} = \begin{bmatrix} \mathbf{A}_{1,1} & \mathbf{A}_{1,2} \\ \mathbf{A}_{2,1} & \mathbf{A}_{2,2} \end{bmatrix}$$

such that $\mathbf{A}_{1,2}$ and $\mathbf{A}_{2,1}$ are of rank at most k and $\mathbf{A}_{1,1}$ and $\mathbf{A}_{2,2}$ are each HODLR(k).

HODLR(k) matrices of size $n \times n$ have $O(kn \log(n))$ parameters. Moreover, given a HODLR factorization, lots of downstream linear algebra tasks can be accelerated.

Our main result

Theorem. There exist matvec algorithms which use $O\big(k\log(n)/\beta^3\big)$ products with $\mathbf{A}$ to obtain a HODLR(k) matrix $\widetilde{\mathbf{A}}$ satisfying

$$\|\mathbf{A}-\widetilde{\mathbf{A}}\|_{\mathsf{F}} \le (1+\beta)^{\log_2(n)} \min_{\mathbf{H}\in\mathrm{HODLR}(k)} \|\mathbf{A}-\mathbf{H}\|_{\mathsf{F}}.$$

Corollary. $(1+\varepsilon)$-optimal approximation with $O\big(k\log(n)^4/\varepsilon^3\big)$ matvecs

Corollary. $n^{0.01}$-optimal approximation with $O\big(k\log(n)\big)$ matvecs

Our main result

Theorem. There exist matvec algorithms which use $O\big(k\log(n)/\beta^3\big)$ products with $\mathbf{A}$ to obtain a HODLR(k) matrix $\widetilde{\mathbf{A}}$ satisfying

$$\|\mathbf{A}-\widetilde{\mathbf{A}}\|_{\mathsf{F}} \le (1+\beta)^{\log_2(n)} \min_{\mathbf{H}\in\mathrm{HODLR}(k)} \|\mathbf{A}-\mathbf{H}\|_{\mathsf{F}}.$$

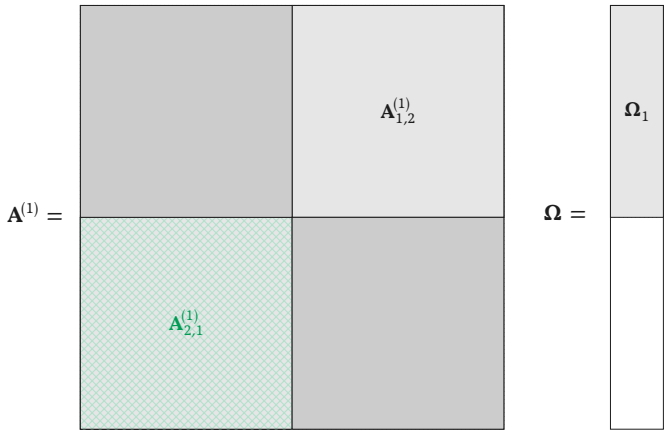
Corollary. $(1+\varepsilon)$-optimal approximation with $O\big(k\log(n)^4/\varepsilon^3\big)$ matvecs

Corollary. $n^{0.01}$-optimal approximation with $O\big(k\log(n)\big)$ matvecs

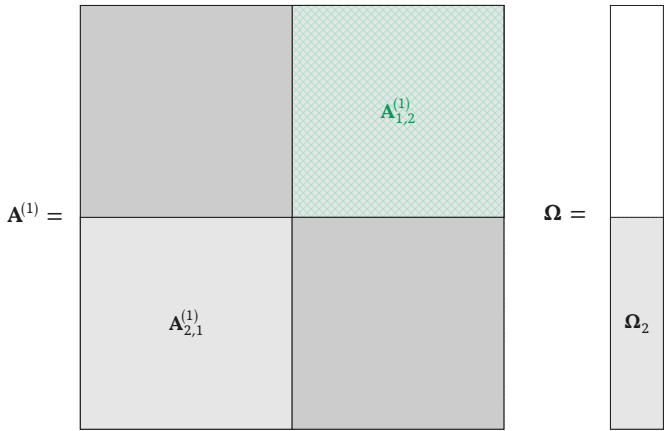
Our approach: robust peeling (idealized case, level 1)

$$\mathbf{A}^{(1)} = \begin{bmatrix} & \mathbf{A}^{(1)}_{1,2} \\ \mathbf{A}^{(1)}_{2,1} & \end{bmatrix}$$

Our approach: robust peeling (idealized case, level 1)

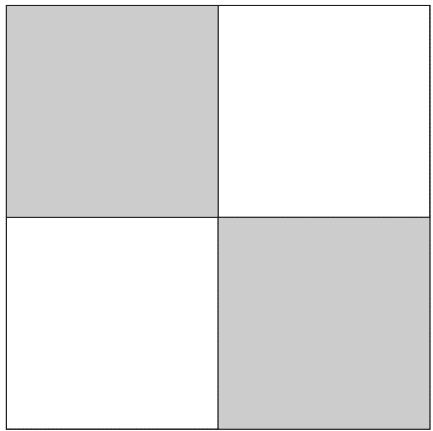


Our approach: robust peeling (idealized case, level 1)



Our approach: robust peeling (idealized case, level 1)

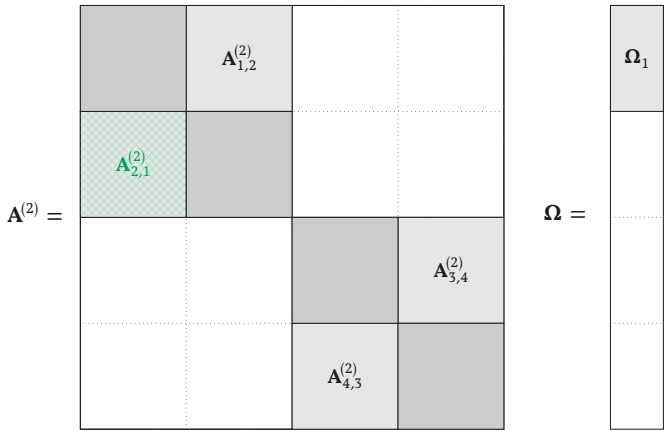
$\mathbf{A}^{(1)} =$



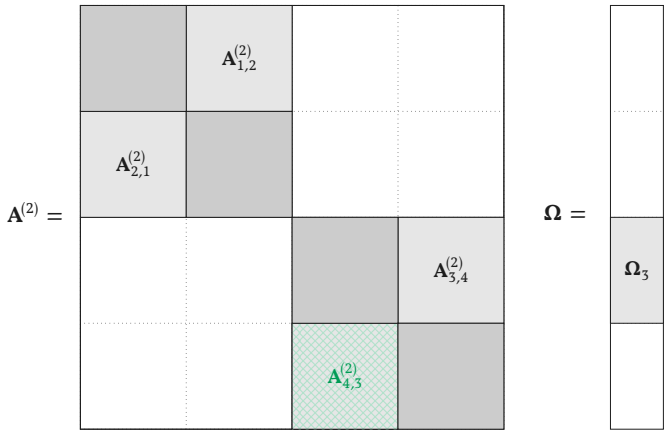
Our approach: robust peeling (idealized case, simultaneous approximation)

$$\mathbf{A}^{(2)} = \begin{bmatrix} & \mathbf{A}^{(2)}_{1,2} & & \\ \mathbf{A}^{(2)}_{2,1} & & & \\ & & & \mathbf{A}^{(2)}_{3,4} \\ & & \mathbf{A}^{(2)}_{4,3} & \end{bmatrix}$$

Our approach: robust peeling (idealized case, simultaneous approximation)

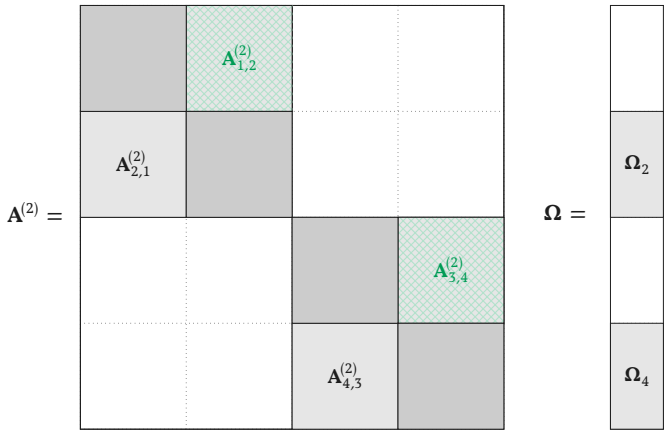


Our approach: robust peeling (idealized case, simultaneous approximation)



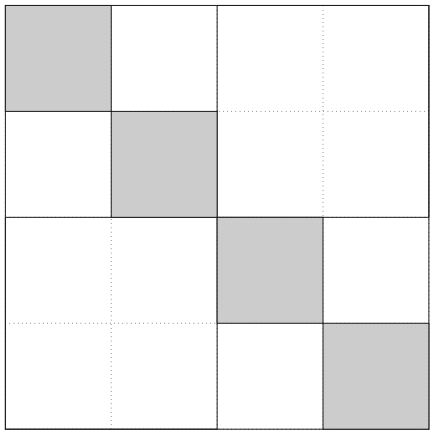
Our approach: robust peeling (idealized case, simultaneous approximation)

Our approach: robust peeling (idealized case, simultaneous approximation)

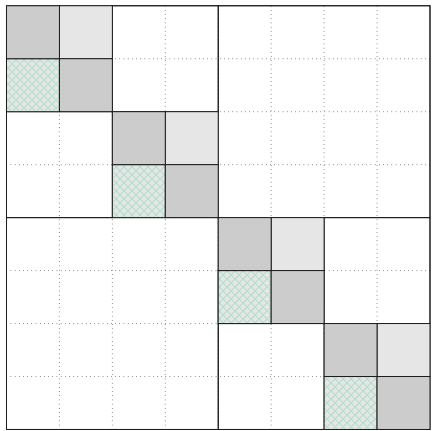


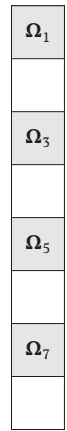
Our approach: robust peeling (idealized case, simultaneous approximation)

$\mathbf{A}^{(2)} =$

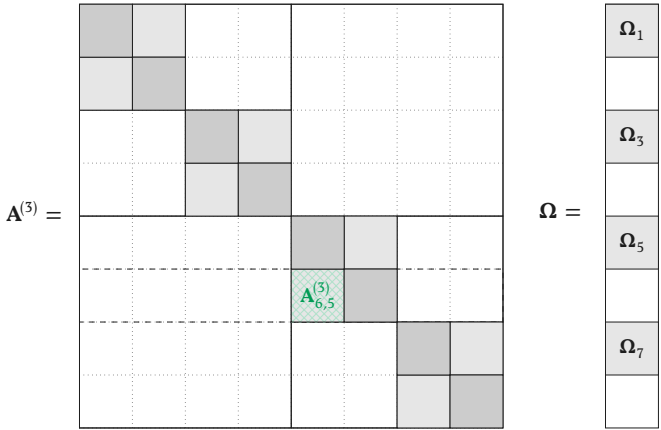


What if there is error?

$\mathbf{A}^{(3)} =$ 

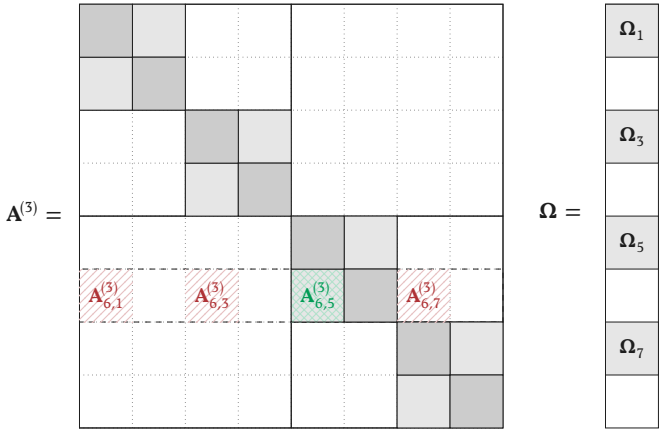
$\boldsymbol{\Omega} =$ 

What if there is error?



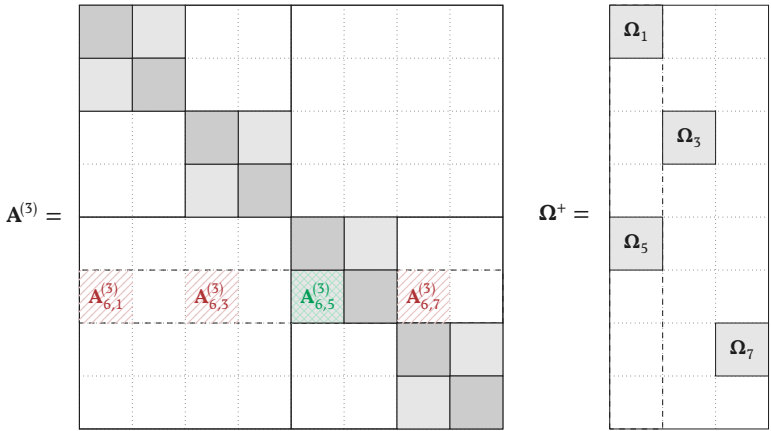
$$\mathbf{A}^{(3)}_{6,5}\boldsymbol{\Omega}_5$$

What if there is error?



$$\mathbf{A}^{(3)}_{6,5}\boldsymbol{\Omega}_5 + \mathbf{A}^{(3)}_{6,1}\boldsymbol{\Omega}_1 + \mathbf{A}^{(3)}_{6,3}\boldsymbol{\Omega}_3 + \mathbf{A}^{(3)}_{6,7}\boldsymbol{\Omega}_7$$

Perforated Block CountSketch



$$\mathbf{A}^{(3)}_{6,5}\mathbf{\Omega}_5 + \mathbf{A}^{(3)}_{6,1}\mathbf{\Omega}_1$$

Proof ingredient: A perturbation bound for the RSVD

Our algorithm is based on ideas like the RSVD and "perforated block count sketch"

- this required cross-disciplinary knowledge to identify and develop key algorithm tools.
- case-study in the use of RandNLA to solve problems in other fields

Our proof relies on technical ingredients including a new perturbation bound for the RSVD (which is likely of independent interest):

Theorem. Let $\mathbf{Q} = \mathrm{orth}(\mathbf{B\Omega} + \mathbf{E}_1)$ and $\mathbf{X} = \mathbf{Q}^\mathsf{T}\mathbf{B} + \mathbf{E}_2$. Then

$$\|\mathbf{B} - \mathbf{Q}[\![\mathbf{X}]\!]_k\|_\mathsf{F} \leq \underbrace{\|\mathbf{E}_1\mathbf{\Omega}_{\mathrm{top}}^\dagger\|_\mathsf{F} + 2\|\mathbf{E}_2\|_\mathsf{F}}_{\text{perturbations}} + \underbrace{\left(\|\mathbf{\Sigma}_{\mathrm{bot}}\|_\mathsf{F}^2 + \|\mathbf{\Sigma}_{\mathrm{bot}}\mathbf{\Omega}_{\mathrm{bot}}\mathbf{\Omega}_{\mathrm{top}}^\dagger\|_\mathsf{F}^2\right)^{1/2}}_{\text{classical RSVD bound}}.$$

Other interesting topics from this line of work

The matrix-vector query model often lets us prove lower-bounds against any matvec algorithm for a given task; i.e. study the complexity of a task!

> **Case study.** For oscillatory operators, butterfly matrices are used. Similar to HODLR matrices, a butterfly matrix has $O(kn\log(n))$ parameters. However, all existing algorithms require $O(\sqrt{n})$ matvecs!! Lot of work to improve this rate..
>
> We have shown a lower-bound of $\Omega(\sqrt{n})$ matvecs!!
>
> – Maybe butterfly isn't the right class for matvec algorithms.

I'm also interested in developing a general theory for matrix-approximation (rather than class-specific algorithms).

– partial progress in Amsel, Avi, et al. 2025

Looking forward: my vision

In the coming years, I will work towards operationalizing RandNLA for widespread use across the sciences.

Operationalizing RandNLA

theoretical algorithm design/analysis

- I will expand the frontier of our understanding of (randomized) NLA algorithms for scientific applications.

problem identification

- My cross-disciplinary background makes me uniquely suited to identify connections between NLA and areas such as quantum physics/chemistry, operator learning, machine learning / data-science.

implementation oriented algorithms

- As RandNLA matures, we must design algorithms with consideration of fine-grained cost tradeoffs relevant to implementation (arithmetic costs, memory, parallelism, communication, …)
  - often depends on architecture (e.g. laptop, GPU clusters, multi-node HPC, …), all of which I'm experienced with
- I will develop stopping criteria, error estimation, and automatic parameter selection, et

Knowledge distillation/accessibility[3]

I really enjoy thinking about how to think about technical subjects. Towards this end, I've worked on several "handbooks":

RandNLA w. Examples

- Online book providing a first introduction to RandNLA, with many accompanying numerical examples.

The Lanczos algorithm for matrix functions: a handbook for scientists

- Accessible introduction to Lanczos-based methods for matrix functions, with a particular emphasis on conceptual understanding of the algorithm in finite precision arithmetic.

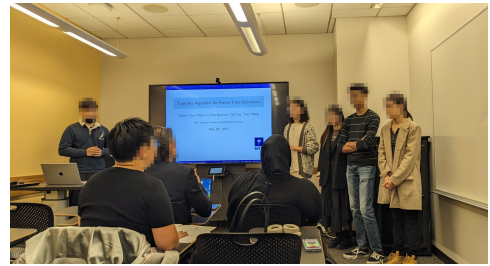
[3] links at `https://research.chen.pw`

Student Research

I've supervised over a dozen independents studies and research projects, including three which lead to publications.[4]

Students were able to present their work:

- Temple Numerical Analysis Day
- SIAM NY-NJ-PA Annual Meeting
- NYU Undergraduate Research Conference
- Alan Edelman's 60th B-day Conference



I'm very excited to continue this work with students at NYU Shanghai!

[4] Xu and Chen 2024; Chen, Chen, et al. 2024; Chen, Huber, Lin, and Zaid 2026.

Thank You

References I

Amsel, Noah, Pratyush Avi, et al. (2025). *Query Efficient Structured Matrix Learning*.

Amsel, Noah, Tyler Chen, et al. (2024). *Fixed-sparsity matrix approximation from matrix-vector products*.

Boullé, Nicolas and Alex Townsend (Jan. 2022). "Learning Elliptic Partial Differential Equations with Randomized Linear Algebra". In: *Foundations of Computational Mathematics* 23.2, pp. 709–739.

Braverman, Mark et al. (Sept. 2020). "The Gradient Complexity of Linear Regression". In: *Proceedings of Thirty Third Conference on Learning Theory*. Ed. by Jacob Abernethy and Shivani Agarwal. Vol. 125. Proceedings of Machine Learning Research. PMLR, pp. 627–647.

Chen, Tyler, Robert Chen, et al. (Nov. 2024). "Faster Randomized Partial Trace Estimation". In: *SIAM Journal on Scientific Computing* 46.6, A3427–A3447.

Chen, Tyler et al. (2026). "Preconditioning without a preconditioner using randomized block Krylov subspace methods". In: *ETNA - Electronic Transactions on Numerical Analysis* 65, pp. 63–92.

Clarkson, Kenneth L. and David P. Woodruff (May 2009). "Numerical linear algebra in the streaming model". In: *Proceedings of the forty-first annual ACM symposium on Theory of computing*. STOC '09. ACM.

Dereziński, Michał, Ethan N. Epperly, and Raphael A. Meyer (2026). *The matrix-vector complexity of* $Ax = b$.

Halko, N., P. G. Martinsson, and J. A. Tropp (2011). "Finding structure with randomness: probabilistic algorithms for constructing approximate matrix decompositions". In: *SIAM Rev.* 53.2, pp. 217–288.

Nakatsukasa, Yuji (2020). "Fast and stable randomized low-rank matrix approximation". In: *ArXiv* abs/2009.11392.

References II

Sarlos, Tamas (Oct. 2006). “Improved Approximation Algorithms for Large Matrices via Random Projections”. In: *2006 47th Annual IEEE Symposium on Foundations of Computer Science (FOCS’06)*. IEEE, pp. 143–152.

Simchowitz, Max, Ahmed El Alaoui, and Benjamin Recht (June 2018). “Tight query complexity lower bounds for PCA via finite sample deformed wigner law”. In: *Proceedings of the 50th Annual ACM SIGACT Symposium on Theory of Computing*. STOC ’18. ACM, pp. 1249–1259.

Tropp, Joel A. and Robert J. Webber (2023). *Randomized algorithms for low-rank matrix approximation: Design, analysis, and applications*.

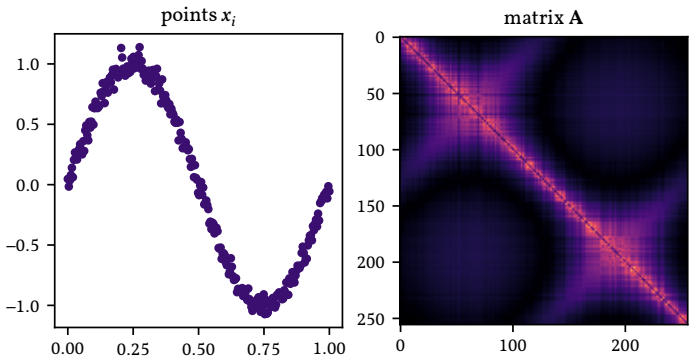
Tropp, Joel A. et al. (Jan. 2017). “Practical Sketching Algorithms for Low-Rank Matrix Approximation”. In: *SIAM Journal on Matrix Analysis and Applications* 38.4, pp. 1454–1485.

Xu, Qichen and Tyler Chen (Apr. 2024). “A posteriori error bounds for the block-Lanczos method for matrix function approximation”. In: *Numerical Algorithms*.

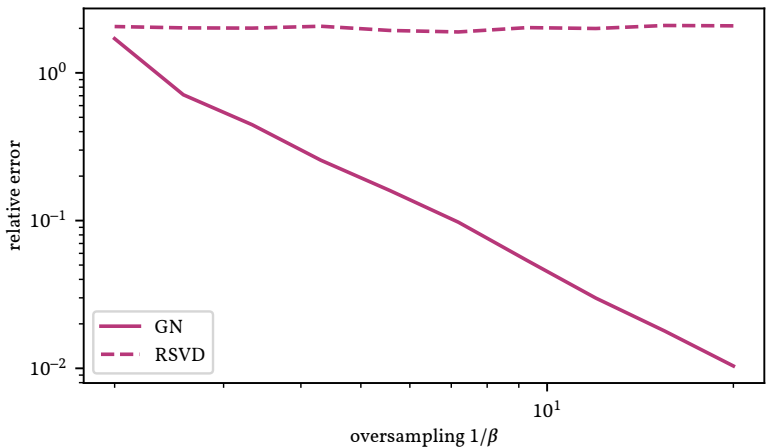
Bonus slides

Numerical experiment

Given points $x_i \in \mathbb{R}^2$, define $[\mathbf{A}]_{i,j} = -\log(\|x_i - x_j\|)$



Numerical experiment



Generalized Nyström[5]

The RSVD tries to compute $\mathbf{Q}^\mathsf{T}\mathbf{B}$ directly; this is the solution to:

$$\min_{\mathbf{X}} \|\mathbf{A} - \mathbf{Q}\mathbf{X}\|_\mathsf{F}.$$

Instead, we can solve a sketched problem:

$$\min_{\mathbf{X}} \|\mathbf{\Psi}^\mathsf{T}\mathbf{A} - \mathbf{\Psi}^\mathsf{T}\mathbf{Q}\mathbf{X}\|_\mathsf{F}.$$

This means $\mathbf{X} = (\mathbf{\Psi}^\mathsf{T}\mathbf{Q})^\dagger \mathbf{\Psi}^\mathsf{T}\mathbf{A}$.

Observation. By adding columns to $\mathbf{\Psi}$, we can damp errors in the product $\mathbf{\Psi}^\mathsf{T}\mathbf{A}$.

The algorithm is also non-adaptive (we can do products with $\mathbf{\Psi}$ in advance)

[5] Clarkson and Woodruff 2009; Tropp, Yurtsever, Udell, and Cevher 2017; Nakatsukasa 2020.

Generalized Nyström[5]

The RSVD tries to compute $\mathbf{Q}^\mathsf{T}\mathbf{B}$ directly; this is the solution to:

$$\min_{\mathbf{X}} \|\mathbf{A} - \mathbf{Q}\mathbf{X}\|_\mathsf{F}.$$

Instead, we can solve a sketched problem:

$$\min_{\mathbf{X}} \|\mathbf{\Psi}^\mathsf{T}\mathbf{A} - \mathbf{\Psi}^\mathsf{T}\mathbf{Q}\mathbf{X}\|_\mathsf{F}.$$

This means $\mathbf{X} = (\mathbf{\Psi}^\mathsf{T}\mathbf{Q})^\dagger \mathbf{\Psi}^\mathsf{T}\mathbf{A}$.

Observation. By adding columns to $\mathbf{\Psi}$, we can damp errors in the product $\mathbf{\Psi}^\mathsf{T}\mathbf{A}$.

The algorithm is also non-adaptive (we can do products with $\mathbf{\Psi}$ in advance)

[5] Clarkson and Woodruff 2009; Tropp, Yurtsever, Udell, and Cevher 2017; Nakatsukasa 2020.

Generalized Nyström[5]

The RSVD tries to compute $\mathbf{Q}^\mathsf{T}\mathbf{B}$ directly; this is the solution to:

$$\min_{\mathbf{X}} \|\mathbf{A} - \mathbf{Q}\mathbf{X}\|_\mathsf{F}.$$

Instead, we can solve a sketched problem:

$$\min_{\mathbf{X}} \|\mathbf{\Psi}^\mathsf{T}\mathbf{A} - \mathbf{\Psi}^\mathsf{T}\mathbf{Q}\mathbf{X}\|_\mathsf{F}.$$

This means $\mathbf{X} = (\mathbf{\Psi}^\mathsf{T}\mathbf{Q})^\dagger \mathbf{\Psi}^\mathsf{T}\mathbf{A}$.

Observation. By adding columns to $\mathbf{\Psi}$, we can damp errors in the product $\mathbf{\Psi}^\mathsf{T}\mathbf{A}$.

The algorithm is also non-adaptive (we can do products with $\mathbf{\Psi}$ in advance)

[5] Clarkson and Woodruff 2009; Tropp, Yurtsever, Udell, and Cevher 2017; Nakatsukasa 2020.

Generalized Nyström (perturbation) analysis

Extend $\mathbf{Q}$ to an orthogonal matrix $[\mathbf{Q}\ \widehat{\mathbf{Q}}]$, and write $\mathbf{\Psi}_1 = \mathbf{\Psi}^\mathsf{T}\mathbf{Q}$ and $\mathbf{\Psi}_2 = \mathbf{\Psi}^\mathsf{T}\widehat{\mathbf{Q}}$.

By orthogonal invariance, $\mathbf{\Psi}_1$ and $\mathbf{\Psi}_2$ are independent Gaussian matrices!

First observe:

$$\mathbf{\Psi}^\mathsf{T}\mathbf{B} = \mathbf{\Psi}^\mathsf{T}(\mathbf{Q}\mathbf{Q}^\mathsf{T} + \widehat{\mathbf{Q}}\widehat{\mathbf{Q}}^\mathsf{T})\mathbf{B} = \mathbf{\Psi}_1\mathbf{Q}^\mathsf{T}\mathbf{B} + \mathbf{\Psi}_2\widehat{\mathbf{Q}}^\mathsf{T}\mathbf{B}.$$

Therefore:

$$\mathbf{X} = (\mathbf{\Psi}^\mathsf{T}\mathbf{Q})^\dagger(\mathbf{\Psi}^\mathsf{T}\mathbf{B}) = \mathbf{\Psi}_1^\dagger\mathbf{\Psi}_1\mathbf{Q}^\mathsf{T}\mathbf{B} + \mathbf{\Psi}_1^\dagger\mathbf{\Psi}_2\widehat{\mathbf{Q}}^\mathsf{T}\mathbf{B} = \mathbf{Q}^\mathsf{T}\mathbf{B} + \mathbf{\Psi}_1^\dagger\mathbf{\Psi}_2\widehat{\mathbf{Q}}^\mathsf{T}\mathbf{B}.$$

Adding more columns to $\mathbf{\Psi}$ (and hence $\mathbf{\Psi}_1$) reduces the error in the second term.

Generalized Nyström (perturbation) analysis

Extend $\mathbf{Q}$ to an orthogonal matrix $[\mathbf{Q}\ \widehat{\mathbf{Q}}]$, and write $\mathbf{\Psi}_1 = \mathbf{\Psi}^\mathsf{T}\mathbf{Q}$ and $\mathbf{\Psi}_2 = \mathbf{\Psi}^\mathsf{T}\widehat{\mathbf{Q}}$.

By orthogonal invariance, $\mathbf{\Psi}_1$ and $\mathbf{\Psi}_2$ are independent Gaussian matrices!

First observe:

$$\mathbf{\Psi}^\mathsf{T}\mathbf{B} + \mathbf{E} = \mathbf{\Psi}^\mathsf{T}(\mathbf{Q}\mathbf{Q}^\mathsf{T} + \widehat{\mathbf{Q}}\widehat{\mathbf{Q}}^\mathsf{T})\mathbf{B} + \mathbf{E} = \mathbf{\Psi}_1\mathbf{Q}^\mathsf{T}\mathbf{B} + \mathbf{\Psi}_2\widehat{\mathbf{Q}}^\mathsf{T}\mathbf{B} + \mathbf{E}.$$

Therefore:

$$\mathbf{X} = (\mathbf{\Psi}^\mathsf{T}\mathbf{Q})^\dagger(\mathbf{\Psi}^\mathsf{T}\mathbf{B} + \mathbf{E}) = \mathbf{\Psi}_1^\dagger\mathbf{\Psi}_1\mathbf{Q}^\mathsf{T}\mathbf{B} + \mathbf{\Psi}_1^\dagger\mathbf{\Psi}_2\widehat{\mathbf{Q}}^\mathsf{T}\mathbf{B} + \mathbf{\Psi}_1^\dagger\mathbf{E} = \mathbf{Q}^\mathsf{T}\mathbf{B} + \mathbf{\Psi}_1^\dagger\mathbf{\Psi}_2\widehat{\mathbf{Q}}^\mathsf{T}\mathbf{B} + \mathbf{\Psi}_1^\dagger\mathbf{E}.$$

Adding more columns to $\mathbf{\Psi}$ (and hence $\mathbf{\Psi}_1$) reduces the error in the second term.

Lower bounds

Our algorithm produces a $(1+\varepsilon)$-optimal approximation using $O(k \operatorname{poly}(\log(n)/\varepsilon))$ matvecs.

This is optimal:

> **Theorem.** There is a constant $C > 0$ such that for any k, n, ε, there exists a (random) matrix $\mathbf{A}$ such that getting a $(1+\varepsilon)$-optimal HODLR approximation requires at least $C\big(k \log_2(n/k) + k/\varepsilon\big)$ matvecs.

Hidden Random Matrix Technique

Our lower bound makes use of a lower-bound I developed in Amsel, Chen, et al. 2024 for *fixed-sparisty* approximation.

> **Proposition.** Fix any sparsity pattern with $\Theta(s)$ nonzero entries in each row and column. There exists a matrix $\mathbf{A}$ such that getting a $(1+\varepsilon)$ approximation of this sparsity requires at least $\Omega(s/\varepsilon)$ matvecs.

Our approach builds on the increasingly popular hidden random matrix technique[6].

[6] Simchowitz, El Alaoui, and Recht 2018; Braverman, Hazan, Simchowitz, and Woodworth 2020; Dereziński, Epperly, and Meyer 2026.

Intuition

Let $\mathbf{a}$ be a length-d Gaussian vector . Consider what happens after we observe $\mathbf{a}^\top \mathbf{x}$ (wlog $\|\mathbf{x}\| = 1$).

Let $\mathbf{X} \in \mathbb{R}^{d \times (d-1)}$ be an orthonormal basis orthogonal to $\mathbf{x}$. Then $\begin{bmatrix} \mathbf{x} & \mathbf{X} \end{bmatrix}$ is orthogonal. Hence, by the orthogonal invariance of the Gaussian distribution

$$\mathbf{a}^\top \begin{bmatrix} \mathbf{x} & \mathbf{X} \end{bmatrix} = \begin{bmatrix} \mathbf{a}^\top \mathbf{x} & \mathbf{a}^\top \mathbf{X} \end{bmatrix}$$

is a length-d Gaussian vector.

We observe $\mathbf{a}^\top \mathbf{x}$, but don't know anything about $\mathbf{a}^\top \mathbf{X} \in \mathbb{R}^{d-1}$.

A lot of randomness remains!

Intuition

Let $\mathbf{a}$ be a length-d Gaussian vector . Consider what happens after we observe $\mathbf{a}^\top \mathbf{x}$ (wlog $\|\mathbf{x}\| = 1$).

Let $\mathbf{X} \in \mathbb{R}^{d \times (d-1)}$ be an orthonormal basis orthogonal to $\mathbf{x}$. Then $\begin{bmatrix} \mathbf{x} & \mathbf{X} \end{bmatrix}$ is orthogonal. Hence, by the orthogonal invariance of the Gaussian distribution

$$\mathbf{a}^\top \begin{bmatrix} \mathbf{x} & \mathbf{X} \end{bmatrix} = \begin{bmatrix} \mathbf{a}^\top \mathbf{x} & \mathbf{a}^\top \mathbf{X} \end{bmatrix}$$

is a length-d Gaussian vector.

We observe $\mathbf{a}^\top \mathbf{x}$, but don't know anything about $\mathbf{a}^\top \mathbf{X} \in \mathbb{R}^{d-1}$.

A lot of randomness remains!

Intuition

Let $\mathbf{a}$ be a length-d Gaussian vector . Consider what happens after we observe $\mathbf{a}^\top \mathbf{x}$ (wlog $\|\mathbf{x}\| = 1$).

Let $\mathbf{X} \in \mathbb{R}^{d \times (d-1)}$ be an orthonormal basis orthogonal to $\mathbf{x}$. Then $\begin{bmatrix} \mathbf{x} & \mathbf{X} \end{bmatrix}$ is orthogonal. Hence, by the orthogonal invariance of the Gaussian distribution

$$\mathbf{a}^\top \begin{bmatrix} \mathbf{x} & \mathbf{X} \end{bmatrix} = \begin{bmatrix} \mathbf{a}^\top \mathbf{x} & \mathbf{a}^\top \mathbf{X} \end{bmatrix}$$

is a length-d Gaussian vector.

We observe $\mathbf{a}^\top \mathbf{x}$, but don't know anything about $\mathbf{a}^\top \mathbf{X} \in \mathbb{R}^{d-1}$.

A lot of randomness remains!